

Pygame Wall Collision Fix

<https://gemini.google.com/app/4f6a58ab652dc732>

User prompt: I have a Pygame project where a marble bounces off empty space near wall corners. Rewrite the `_resolve_wall_collisions` method in `game_engine.py` to use a true circle-vs-rectangle distance test instead of `colliderect`. Find the closest point on the wall's rectangle to the marble's center, calculate the distance, and adjust the position/velocity only if the distance is less than the marble's radius.

Response: Replace the existing `_resolve_wall_collisions` method in `game_engine.py` with the following code. This implementation calculates the exact closest point on the wall's rectangle to the marble's center, ensuring bounces only occur when the round marble visually touches the wall, preventing the "phantom" corner bounces.

```
def _resolve_wall_collisions(self):
    for wall in self.walls:
        wall_rect = wall.rect()

        # Find the closest point on the wall's rectangle to the marble's center
        closest_x = max(wall_rect.left, min(self.marble.x, wall_rect.right))
        closest_y = max(wall_rect.top, min(self.marble.y, wall_rect.bottom))

        # Calculate distance components from the center to the closest point
        dx = self.marble.x - closest_x
        dy = self.marble.y - closest_y
        distance_squared = dx ** 2 + dy ** 2

        # Check if the marble is colliding with the wall (distance < radius)
        if distance_squared < self.marble.radius ** 2:
            distance = distance_squared ** 0.5

            # Failsafe: if distance is exactly 0, the marble's center is inside the wall
            if distance == 0:
                self.marble.y -= self.marble.radius
                self.marble.vy *= -0.3
                continue

            # Calculate the overlap amount and the normalized direction vector (normal)
            overlap = self.marble.radius - distance
            nx = dx / distance
            ny = dy / distance

            # Push the marble out of the wall along the normal vector
            self.marble.x += nx * overlap
            self.marble.y += ny * overlap

            # Calculate the dot product of the velocity and the collision normal
            dot_product = self.marble.vx * nx + self.marble.vy * ny

            # Apply velocity reflection only if the marble is moving towards the wall
            if dot_product < 0:
                # Use a restitution of 0.3 to match the original bounce logic
                restitution = 0.3
                impulse = (1 + restitution) * dot_product

                self.marble.vx -= impulse * nx
                self.marble.vy -= impulse * ny
```

Key Changes

- **True Distance Calculation:** Replaces `marble_rect.colliderect(wall_rect)` with Pythagorean distance to the closest clamped edge of the wall.
- **Vector-Based Reflection:** Instead of simply flipping velocity based on whether horizontal or vertical overlap was larger, this reflects the marble's velocity across the collision normal vector ($V = V - (1 + \text{restitution})(V \cdot N)N$).
- **Corner Accuracy:** By calculating a normal vector (nx, ny), the marble now bounces realistically off of 90-degree corners at an angle, whereas the old logic strictly flipped vx or vy.

User prompt: Add a replay feature to `game_engine.py`. During the game over screen, display 'Press 1 (Easy), 2 (Med), 3 (Hard) to Replay'. Add a `reset(difficulty)` method that resets the marble to (50,50), resets `start_ticks`, sets `game_over` to False, and alters `tilt_strength`, `friction`, and `time_limit_ms` based on the difficulty. Handle these keystrokes in `handle_event`

Response: Add the following `reset` method to your `GameEngine` class in `game_engine.py` to handle resetting the state and adjusting variables based on the chosen difficulty.

```
def reset(self, difficulty):
    # Reset marble position and velocity
    self.marble.x = 50
    self.marble.y = 50
    self.marble.vx = 0
    self.marble.vy = 0

    # Reset game state and timers
    self.game_over = False
    self.result = None
    self.finish_time_ms = None
    self.start_ticks = pygame.time.get_ticks()

    # Apply difficulty settings
    if difficulty == 1: # Easy
        self.tilt_strength = 0.8
        self.friction = 0.04
        self.time_limit_ms = 60000
    elif difficulty == 2: # Medium
        self.tilt_strength = 0.6
        self.friction = 0.02
        self.time_limit_ms = 45000
    elif difficulty == 3: # Hard
        self.tilt_strength = 0.4
        self.friction = 0.01
        self.time_limit_ms = 30000
```

Next, update the `handle_event` method to listen for the 1, 2, and 3 keys when the game is over:

```
def handle_event(self, event):
    if self.game_over and event.type == pygame.KEYDOWN:
        if event.key == pygame.K_1:
            self.reset(1)
        elif event.key == pygame.K_2:
            self.reset(2)
        elif event.key == pygame.K_3:
            self.reset(3)
```

Finally, replace the existing `if self.game_over:` console print block at the end of the `render` method with Pygame text rendering to visually display the game over state and the replay instructions:

```
def render(self, screen):
    screen.fill(DARK)

    for wall in self.walls:
        pygame.draw.rect(screen, WALL_COLOR, wall.rect())

    pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
    pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)), self.marble.radius)

    elapsed = pygame.time.get_ticks() - self.start_ticks
    seconds_left = max(0, (self.time_limit_ms - elapsed) // 1000)
    timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
    screen.blit(timer_text, (10, 10))

    # Replaced the console print block with on-screen text rendering
    if self.game_over:
        if self.result == "solved":
            status_msg = f"Solved! Finished in {self.finish_time_ms / 1000:.1f}s"
        else:
            status_msg = "Time's up! Maze not solved."

        status_text = self.font.render(status_msg, True, WHITE)
        replay_text = self.font.render("Press 1 (Easy), 2 (Med), 3 (Hard) to Replay", True, WHITE)

        # Center the text on the screen
        screen.blit(status_text, (self.width // 2 - status_text.get_width() // 2, self.height // 2 - 30))
        screen.blit(replay_text, (self.width // 2 - replay_text.get_width() // 2, self.height // 2 + 10))
```

User prompt: Update `game_engine.py` to stop using `print()` for game over states. Add logic to the `render` method to display a 'Solved! Time: [X]s' text in green if `self.result == 'solved'`, or 'Time's up!' in red if `self.result == 'timeout'`. Wait for user input on this screen.

Response: Update the render method in `game_engine.py` to use specific text colors based on the win/loss condition instead of printing to the console. This builds on the replay text added previously.

```
def render(self, screen):
    screen.fill(DARK)

    for wall in self.walls:
        pygame.draw.rect(screen, WALL_COLOR, wall.rect())

    pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
    pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)), self.marble.radius)

    elapsed = pygame.time.get_ticks() - self.start_ticks
    seconds_left = max(0, (self.time_limit_ms - elapsed) // 1000)
    timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
    screen.blit(timer_text, (10, 10))

    if self.game_over:
        if self.result == "solved":
            # Display solved message in Green (matching GOAL_COLOR)
            status_text = self.font.render(f"Solved! Time: {self.finish_time_ms / 1000:.1f}s", True, (60, 200, 120))
        else:
            # Display timeout message in Red
            status_text = self.font.render("Time's up!", True, (220, 60, 60))

        replay_text = self.font.render("Press 1 (Easy), 2 (Med), 3 (Hard) to Replay", True, WHITE)

        # Center the text on the screen
        screen.blit(status_text, (self.width // 2 - status_text.get_width() // 2, self.height // 2 - 30))
        screen.blit(replay_text, (self.width // 2 - replay_text.get_width() // 2, self.height // 2 + 10))
```

Because the update method stops modifying the marble's position when `self.game_over` is `True` and the `handle_event` method from the previous step now listens for the 1, 2, and 3 keys during the game-over state, the game gracefully pauses and waits for user input as required.

User prompt: Initialize pygame.mixer in game_engine.py. Load three sound files (bounce.wav, win.wav, lose.wav). Trigger the bounce sound inside _resolve_wall_collisions when a collision occurs. Trigger the win sound when reaching the goal, and the lose sound when timing out in the update method. Ensure they only play once per event

Response: To add sound feedback, initialize `pygame.mixer` in the `__init__` method, load the sounds, and trigger them at the correct state transitions.

Here are the required updates to `game_engine.py`:

1. Update `__init__` to load sounds

```
def __init__(self, width, height):
    self.width = width
    self.height = height

    self.marble = Marble(50, 50)
    self.tilt_strength = 0.6
    self.friction = 0.02
    self.max_speed = 9

    self.walls = self._build_maze()
    self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22

    self.time_limit_ms = 45000
    self.start_ticks = pygame.time.get_ticks()

    self.font = pygame.font.SysFont("Arial", 26)
    self.game_over = False
    self.result = None
    self.finish_time_ms = None

    # Initialize mixer and load sounds
    pygame.mixer.init()
    try:
        self.bounce_snd = pygame.mixer.Sound("bounce.wav")
        self.win_snd = pygame.mixer.Sound("win.wav")
        self.lose_snd = pygame.mixer.Sound("lose.wav")
    except FileNotFoundError:
```

```
# Failsafe in case audio files are missing from the directory
self.bounce_snd = self.win_snd = self.lose_snd = None
```

2. Update update to trigger Win/Lose sounds

Because `self.game_over = True` prevents the update logic from running on subsequent frames, placing the sound triggers inside the win/loss condition blocks guarantees they only play exactly once per round.

```
def update(self):
    if self.game_over:
        return

    elapsed = pygame.time.get_ticks() - self.start_ticks
    if elapsed >= self.time_limit_ms:
        self.game_over = True
        self.result = "timeout"
        if self.lose_snd: self.lose_snd.play()
        return

    self.marble.vx *= (1 - self.friction)
    self.marble.vy *= (1 - self.friction)

    speed = (self.marble.vx ** 2 + self.marble.vy ** 2) ** 0.5
    if speed > self.max_speed:
        scale = self.max_speed / speed
        self.marble.vx *= scale
        self.marble.vy *= scale

    self.marble.x += self.marble.vx
    self.marble.y += self.marble.vy

    self._resolve_wall_collisions()

    gx = self.goal_x - self.marble.x
    gy = self.goal_y - self.marble.y
    if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
        self.game_over = True
        self.result = "solved"
        self.finish_time_ms = elapsed
        if self.win_snd: self.win_snd.play()
```

3. Update _resolve_wall_collisions to trigger Bounce sound

Trigger the sound right before applying the velocity reflection. Because we only reflect velocity if `dot_product < 0` (meaning the marble is moving *towards* the wall), the sound is naturally prevented from triggering multiple times while the marble slides against a surface.

```
def _resolve_wall_collisions(self):
    for wall in self.walls:
        wall_rect = wall.rect()

        closest_x = max(wall_rect.left, min(self.marble.x, wall_rect.right))
        closest_y = max(wall_rect.top, min(self.marble.y, wall_rect.bottom))

        dx = self.marble.x - closest_x
        dy = self.marble.y - closest_y
        distance_squared = dx ** 2 + dy ** 2

        if distance_squared < self.marble.radius ** 2:
            distance = distance_squared ** 0.5

            if distance == 0:
                self.marble.y -= self.marble.radius
                self.marble.vy *= -0.3
                if self.bounce_snd: self.bounce_snd.play()
                continue

            overlap = self.marble.radius - distance
            nx = dx / distance
            ny = dy / distance

            self.marble.x += nx * overlap
            self.marble.y += ny * overlap

            dot_product = self.marble.vx * nx + self.marble.vy * ny
```

```
if dot_product < 0:  
    if self.bounce_snd: self.bounce_snd.play()  
  
    restitution = 0.3  
    impulse = (1 + restitution) * dot_product  
  
    self.marble.vx -= impulse * nx  
    self.marble.vy -= impulse * ny
```
